KitchenLang IDE: Official User Manual

For ease of access, Access our **KitchenLang by clicking this link**

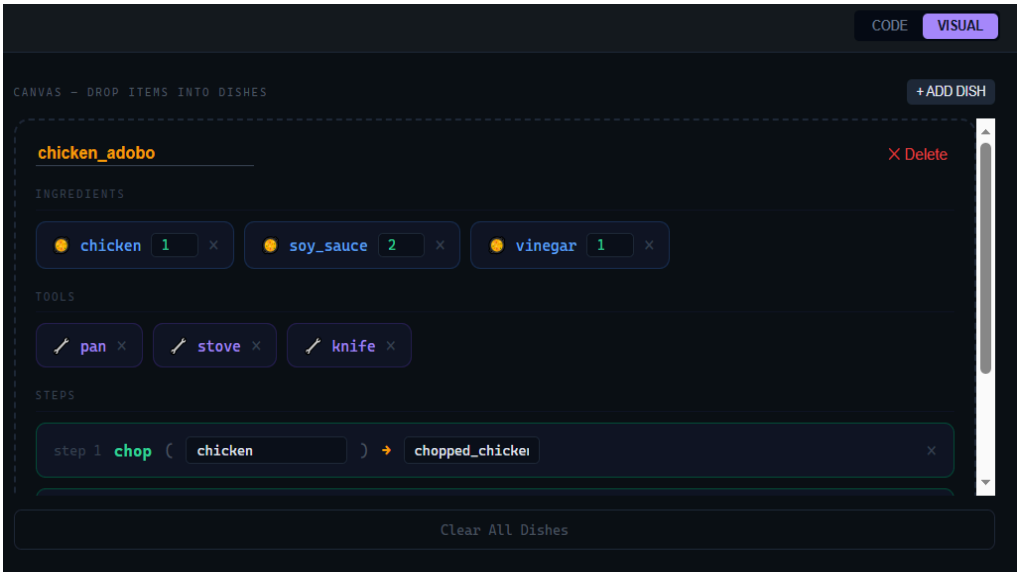
Welcome to the KitchenLang IDE! This integrated development environment allows you to write, visualize, compile, and simulate culinary recipes using a custom programming language.

1. Interface Overview

The IDE is divided into three main workspaces:

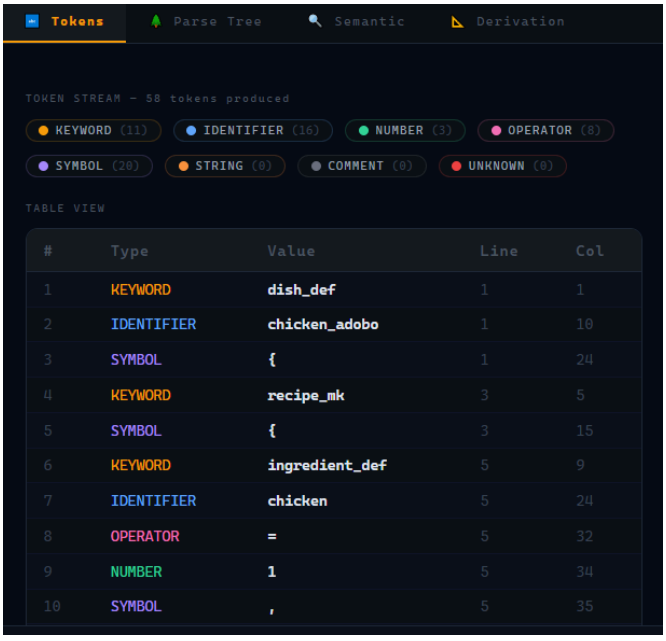
Left Pane (The Editor): Toggle between Code Mode (a text editor with live syntax highlighting) and Visual Mode (a drag-and-drop canvas). Both modes automatically sync with each other.

Code Mode

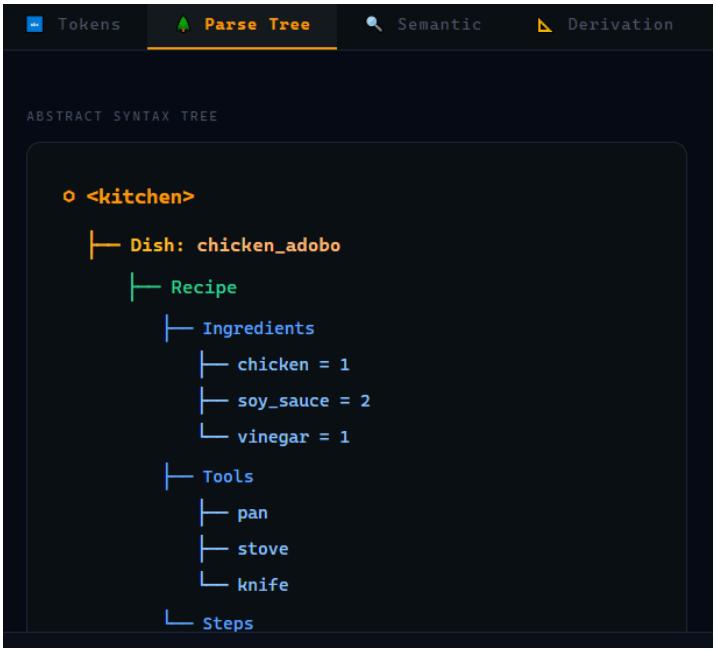


Visual Mode

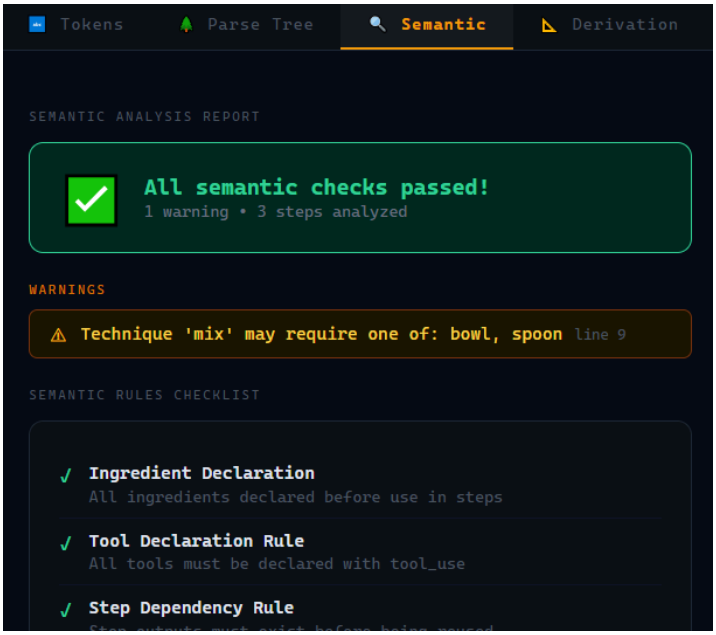
Right Pane (The Compiler Pipeline):
Inspect how the compiler understands your code. Features tabs for Tokenization, the Abstract Syntax Tree (AST), Semantic Analysis, and CFG Derivations.



Tokens Tab



Parse Tree Tab

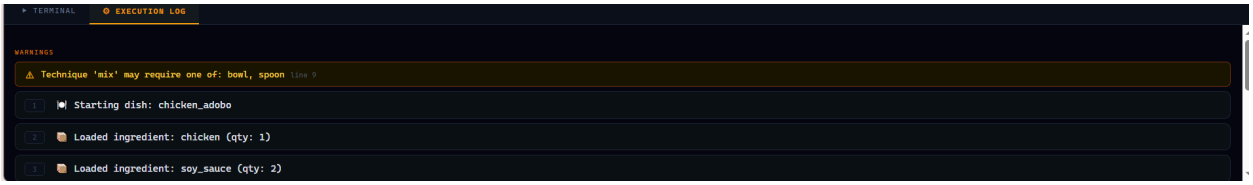


Semantic Tab

DerivationTab

Bottom Pane (The Terminal and Execution Log): Watch the Virtual Machine execute your recipe step-by-step. If your code asks for user input, you will type it here.

Terminal



Execution Log

2. Writing KitchenLang (Language Syntax)

Every KitchenLang script must follow a strict structural hierarchy.

BASIC STRUCTURE

```
dish_def dish_name {
    recipe_mk {
        ingredient_def ingredient1 = 1, ingredient2 = 2;
        tool_use tool1, tool2;

        step_do technique {
            output_item = input_item1 + input_item2;
        }
    }
    serve_out final_output;
}
```

Keyword	Purpose	Example
dish_def	Defines the start of a new dish and its name.	dish_def pasta_salad {
recipe_mk	Opens the main recipe block.	recipe_mk {
ingredient_def	Declares raw ingredients and quantities.	ingredient_def salt = 1;
input()	Prompts the user for a dynamic quantity.	water = input("How much?");
tool_use	Declares tools required for the dish.	tool_use pot, pan;
step_do	Performs a culinary technique.	step_do boil {
output()	Prints a message or ingredient to the terminal.	output("Done!");

serve_out	Ends the dish and serves the final product.	serve_out cooked_pasta;

3. Top Bar Actions

TUTORIAL

Launches an interactive, step-by-step spotlight tour of the IDE's features.

Tutorial Mode

FORMAT CODE

Reads your compiled AST and automatically reformats your messy text into perfectly indented, standardized code. *(Requires a successful compilation first).*

```
dish_def chicken_adobo {

    recipe_
mk {

        ingredient_def chi
cken = 1, soy_sauce = 2, vinegar = 1;

        tool_use pan, stove, knife;

        step_do chop {
            chopped_chicken =
 chicken;
        }

        step_do mix {
            marinade = chopped_chicken + soy_sauce + vinegar;
        }
```

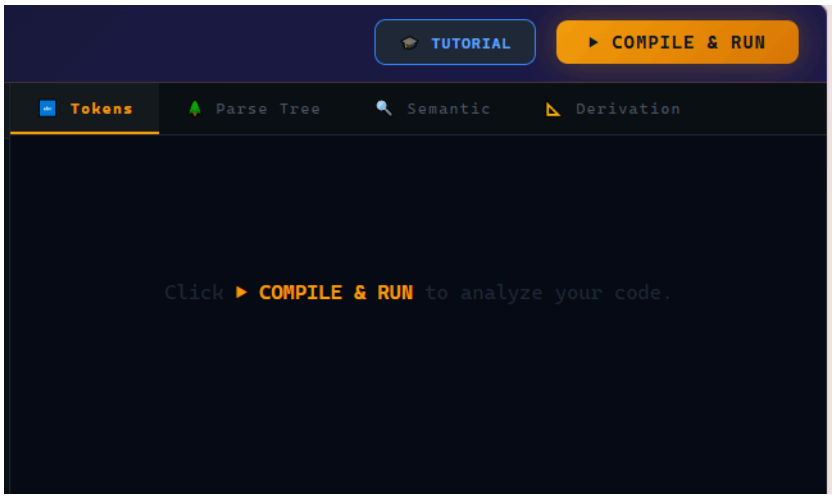
Original Code

```
dish_def chicken_adobo {

    recipe_mk {

        ingredient_def chicken = 1, soy_sauce = 2, vinegar = 1;

        tool_use pan, stove, knife;

        step_do chop {
            chopped_chicken = chicken;
        }

        step_do mix {
            marinade = chopped_chicken + soy_sauce + vinegar;
        }

        step_do fry {
            cooked_dish = marinade;
```

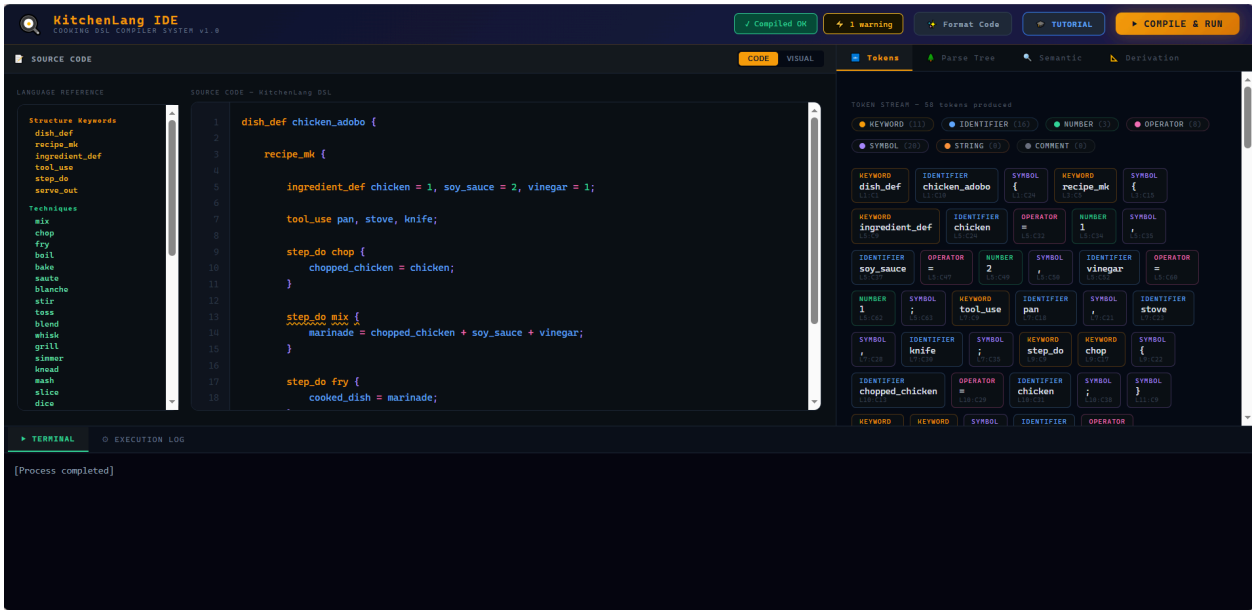
Formatted Code

COMPILE & RUN

Processes your code through the Lexer, Parser, and Semantic Analyzer. If no errors are found, it triggers the Virtual Machine to execute your dish in the bottom terminal.



Compile & Run Button

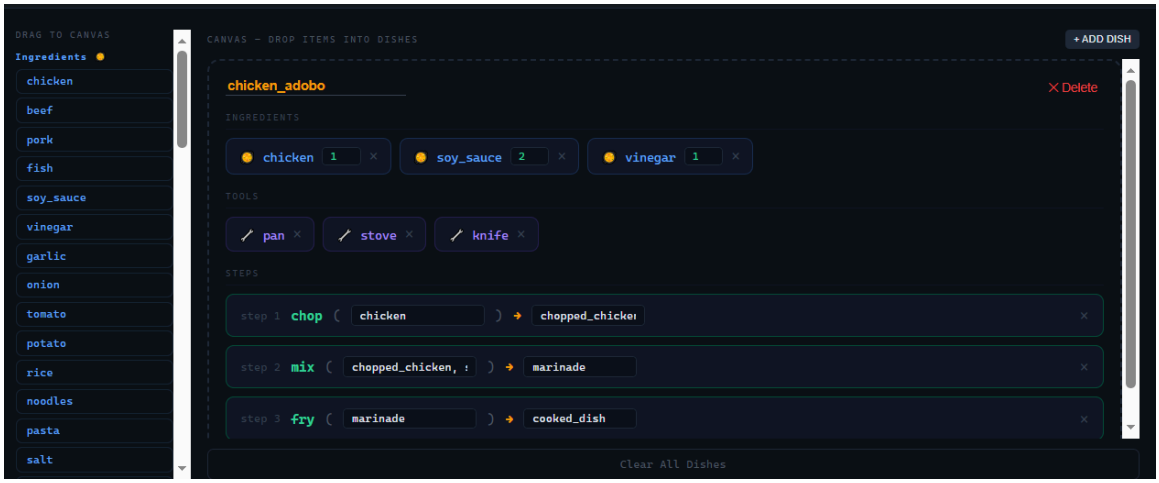


Compiled Code

4. Visual Mode (No-Code Editing)

If you prefer not to type, switch to Visual Mode.

1. Click **+ ADD DISH** to create a new canvas.
2. Drag **Ingredients**, **Tools**, and **Techniques** from the left palette onto your dish canvas.
3. Fill in the input fields (quantities for ingredients, arguments for steps).
4. Switch back to **Code Mode**, and your visual layout will automatically generate perfect KitchenLang code!

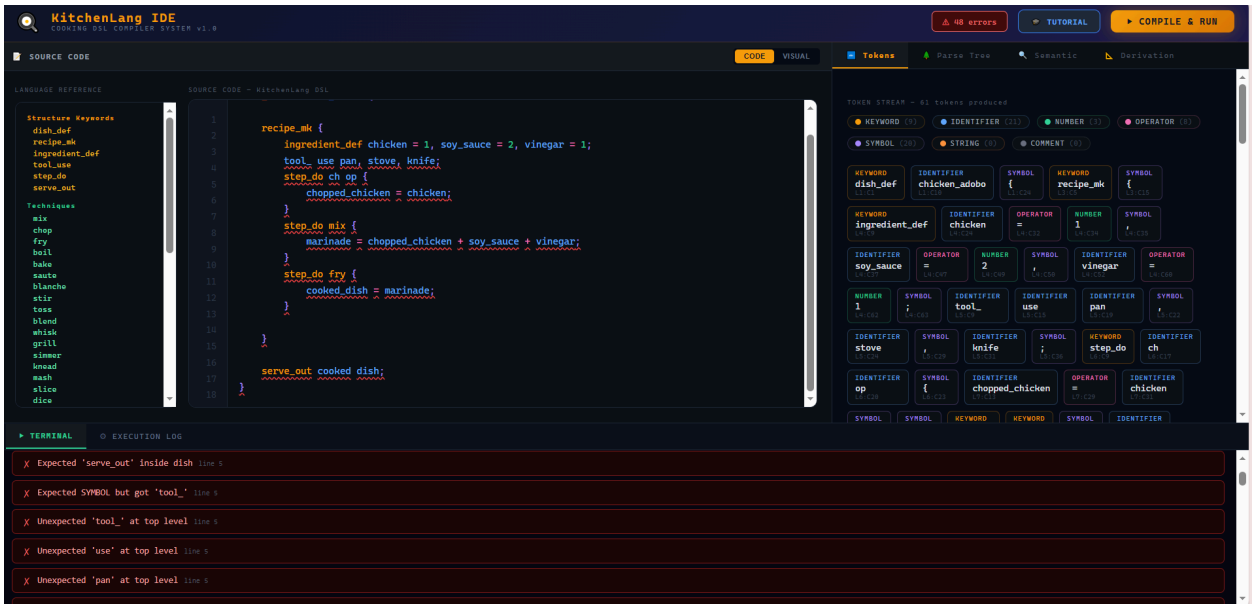


Visual Mode

5. Semantic Rules & Reminders

The compiler's Semantic Analyzer is strict. Keep these rules in mind to avoid errors:

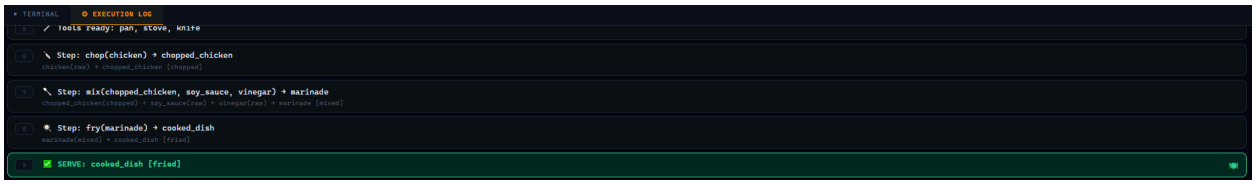
- **Declaration Before Use:** You cannot use an ingredient in a `step_do` unless it was declared in `ingredient_def` or created as an output of a previous step.
- **Linear Consumption:** Ingredients are consumed when used! If you use `chopped_veg` in a `boil` step, you cannot use `chopped_veg` again in a later step. It has been transformed.
- **Tool Requirements:** Certain techniques require specific tools. If you use `step_do blend`, you *must* have declared `tool_use blender;`. If you forget, the compiler will issue a **Warning**.
- **Serve Output:** The variable you pass to `serve_out` must actually exist (either as a raw ingredient or the result of a step).
- **Comments:** Use # to write comments in your code. Note that the **Format Code** button strips comments because it regenerates code from the raw Syntax Tree.



Sample

6. Execution Logging

After clicking Compile & Run, open the **Terminal** tab to see your raw outputs and prompts. Switch to the **Execution Log** tab to see a beautiful, emoji-rich timeline of your recipe being prepared!



Execution Log